MVC Driven Web Application using Spring Boot and Hibernate

Aim

To develop a web-based application using Spring Boot framework following MVC architecture and perform database operations using Hibernate (JPA).

Description

This project implements an MVC-based application using Spring Boot. The controller layer handles HTTP requests, the service layer processes business logic, and the model layer represents database entities. Hibernate (JPA) is used for object-relational mapping and database interaction. The application connects to a MySQL database using JDBC and performs operations on file metadata such as storing and retrieving records.

MVC Layers with File Names

Controller Layer

VaultController.java

```
package com.securevault.controller;

import com.securevault.dto.FileRequestDTO;
import com.securevault.dto.FileResponseDTO;
import com.securevault.dto.VerifyResponseDTO;
import com.securevault.model.AuditLog;
import com.securevault.service.AuditService;
import com.securevault.service.VaultService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api")
```

```
@CrossOrigin(origins = "*")
public class VaultController {

    @Autowired
    private VaultService vaultService;

    @Autowired
    private AuditService auditService;

    @PostMapping("/files/add")
    public ResponseEntity<FileResponseDTO> addFile(@RequestBody FileRequestDTO request,
                                                   @RequestHeader("username") String username) {
        FileResponseDTO response = vaultService.addFile(request, username);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/files/all")
    public ResponseEntity<List<FileResponseDTO>> getAllFiles() {
        List<FileResponseDTO> files = vaultService.getAllFiles();
        return ResponseEntity.ok(files);
    }

    @PostMapping("/files/verify/{id}")
    public ResponseEntity<VerifyResponseDTO> verifyFile(@PathVariable Long id,
                                                        @RequestHeader("username") String username) {
        VerifyResponseDTO response = vaultService.verifyFile(id, username);
        return ResponseEntity.ok(response);
```

```
    }

    @GetMapping("/audit/all")
    public ResponseEntity<List<AuditLog>> getAllAuditLogs() {
        List<AuditLog> logs = auditService.getAllAuditLogs();
        return ResponseEntity.ok(logs);
    }
}
```

VaultController.java

Source History

```java
package com.securevault.controller;

import com.securevault.dto.FileRequestDTO;
import com.securevault.dto.FileResponseDTO;
import com.securevault.dto.VerifyResponseDTO;
import com.securevault.model.AuditLog;
import com.securevault.service.AuditService;
import com.securevault.service.VaultService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api")
@CrossOrigin(origins = "*")
public class VaultController {

    @Autowired
    private VaultService vaultService;

    @Autowired
    private AuditService auditService;

    @PostMapping("/files/add")
    public ResponseEntity<FileResponseDTO> addFile(@RequestBody FileRequestDTO request,
                                                   @RequestHeader("username") String username) {
        FileResponseDTO response = vaultService.addFile(request, username);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/files/all")
    public ResponseEntity<List<FileResponseDTO>> getAllFiles() {
        List<FileResponseDTO> files = vaultService.getAllFiles();
        return ResponseEntity.ok(files);
    }

    @PostMapping("/files/verify/{id}")
    public ResponseEntity<VerifyResponseDTO> verifyFile(@PathVariable Long id,
                                                        @RequestHeader("username") String username) {
        VerifyResponseDTO response = vaultService.verifyFile(id, username);
        return ResponseEntity.ok(response);
    }

    @GetMapping("/audit/all")
    public ResponseEntity<List<AuditLog>> getAllAuditLogs() {
        List<AuditLog> logs = auditService.getAllAuditLogs();
        return ResponseEntity.ok(logs);
    }
}
```

Service Layer

VaultService.java

```
package com.securevault.service;

import com.securevault.dto.FileRequestDTO;
import com.securevault.dto.FileResponseDTO;
import com.securevault.dto.VerifyResponseDTO;
import com.securevault.model.FileMetadata;
import com.securevault.repository.FileMetadataRepository;
import com.securevault.util.DateUtil;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;
import java.util.stream.Collectors;

@Service
public class VaultService {

    @Autowired
    private FileMetadataRepository fileMetadataRepository;

    @Autowired
    private HashService hashService;

    @Autowired
    private RiskAnalysisService riskAnalysisService;
```

```
@Autowired
private AuditService auditService;

@Transactional
public FileResponseDTO addFile(FileRequestDTO request, String username) {
    // Generate hash
    String hash = hashService.generateFileHash(
        request.getFileName(),
        request.getOwner(),
        request.getSize()
    );

    // Analyze risk
    String riskLevel = riskAnalysisService.analyzeRiskLevel(request.getFileName());

    // Create file metadata
    FileMetadata metadata = new FileMetadata(
        request.getFileName(),
        request.getOwner(),
        request.getFileType(),
        request.getSize(),
        DateUtil.getCurrentDateTime(),
        hash,
        riskLevel
    );

    FileMetadata saved = fileMetadataRepository.save(metadata);

    // Log action
```

```
    auditService.logAction(username, "Added file: " + request.getFileName() +
        " (Risk: " + riskLevel + ")");

    return convertToDTO(saved);
}

public List<FileResponseDTO> getAllFiles() {
    return fileMetadataRepository.findAll().stream()
        .map(this::convertToDTO)
        .collect(Collectors.toList());
}

public VerifyResponseDTO verifyFile(Long fileId, String username) {
    FileMetadata file = fileMetadataRepository.findById(fileId)
        .orElseThrow(() -> new RuntimeException("File not found with id: " + fileId));

    // Recalculate hash using the same method
    String currentHash = hashService.generateFileHash(
        file.getFileName(),
        file.getOwner(),
        file.getSize()
    );

    String status = currentHash.equals(file.getHashValue()) ? "VERIFIED" : "TAMPERED";
    String message = status.equals("VERIFIED") ?
        "File integrity is intact" :
        "File has been modified!";

    // Log action
```

```
        auditService.logAction(username, "Verified file: " + file.getFileName() +
            " - Status: " + status);

        return new VerifyResponseDTO(
            fileId,
            file.getFileName(),
            file.getHashValue(),
            currentHash,
            status,
            message
        );
    }

    private FileResponseDTO convertToDTO(FileMetadata metadata) {
        return new FileResponseDTO(
            metadata.getId(),
            metadata.getFileName(),
            metadata.getOwner(),
            metadata.getFileType(),
            metadata.getSize(),
            metadata.getUploadDate(),
            metadata.getHashValue(),
            metadata.getRiskLevel()
        );
    }
}
```

VaultController.java × VaultService.java ×

Source History

```java
    private FileMetadataRepository fileMetadataRepository;

    @Autowired
    private HashService hashService;

    @Autowired
    private RiskAnalysisService riskAnalysisService;

    @Autowired
    private AuditService auditService;

    @Transactional
    public FileResponseDTO addFile(FileRequestDTO request, String username) {
        // Generate hash
        String hash = hashService.generateFileHash(
            request.getFileName(),
            request.getOwner(),
            request.getSize()
        );

        // Analyze risk
        String riskLevel = riskAnalysisService.analyzeRiskLevel(request.getFileName());

        // Create file metadata
        FileMetadata metadata = new FileMetadata(
            request.getFileName(),
            request.getOwner(),
            request.getFileType(),
            request.getSize(),
            DateUtil.getCurrentDateTime(),
            hash,
            riskLevel
        );

        FileMetadata saved = fileMetadataRepository.save(metadata);

        // Log action
        auditService.logAction(username, "Added file: " + request.getFileName() +
            " (Risk: " + riskLevel + ")");

        return convertToDTO(saved);
    }

```

Repository Layer

FileMetadataRepository.java

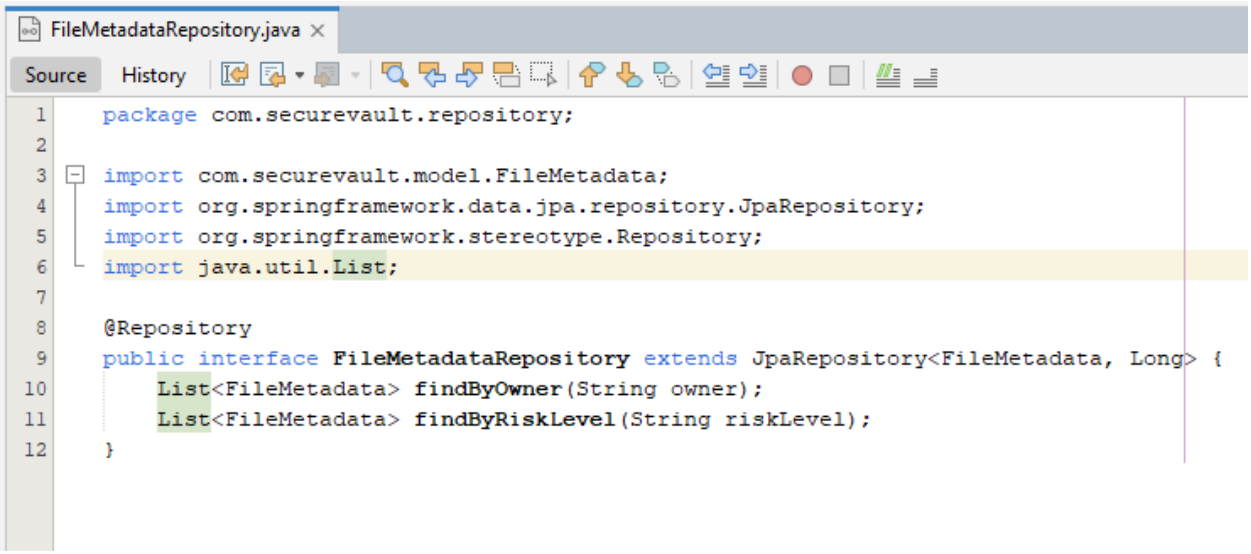
package com.securevault.repository;

import com.securevault.model.FileMetadata;

import org.springframework.data.jpa.repository.JpaRepository;

```
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface FileMetadataRepository extends JpaRepository<FileMetadata, Long> {
    List<FileMetadata> findByOwner(String owner);
    List<FileMetadata> findByRiskLevel(String riskLevel);
}
```



Model Layer

FileMetadata.java

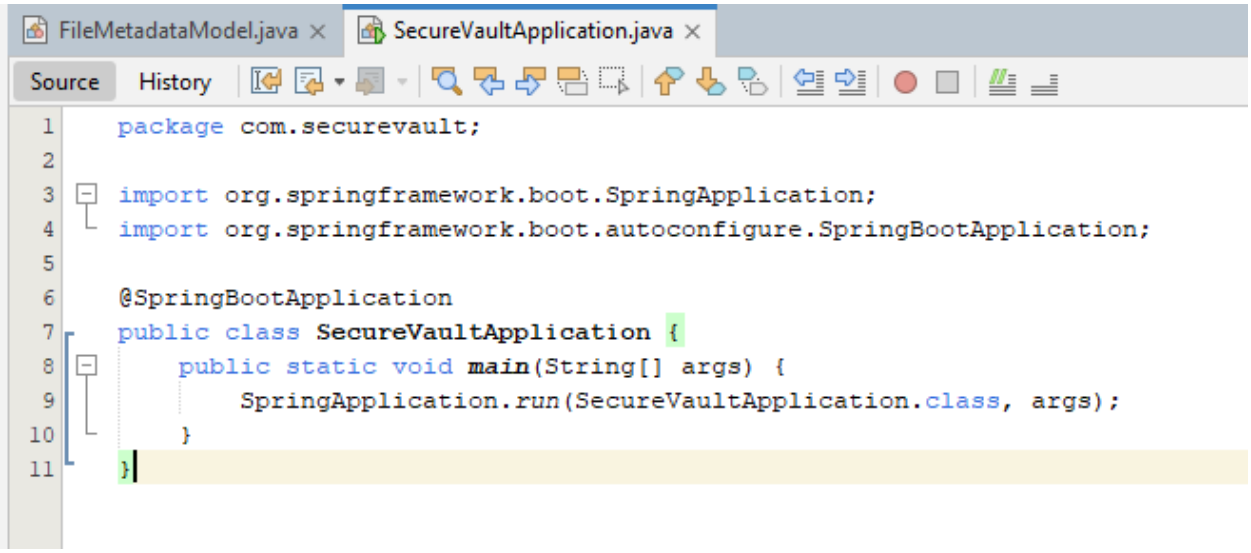
Main Application

SecureVaultApplication.java

```
package com.securevault;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
```

```
@SpringBootApplication
public class SecureVaultApplication {
    public static void main(String[] args) {
        SpringApplication.run(SecureVaultApplication.class, args);
    }
}
```



Configuration

application.properties

```
# Server Configuration
server.port=8081

# Database Configuration
spring.datasource.url=jdbc:mysql://localhost:3306/securevault?createDatabaseIfNotExist=true&useSSL=false&allowPublicKeyRetrieval=true&serverTimezone=UTC
spring.datasource.username=root
spring.datasource.password=114114
```

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

JPA Configuration

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect

spring.jpa.properties.hibernate.format_sql=true

Logging

logging.level.org.springframework.web=INFO

logging.level.com.securevault=DEBUG

Search Results × FileMetadataRepository.java × application.properties ×

Source History

```
# Server Configuration
server.port=8081

# Database Configuration
spring.datasource.url=jdbc:mysql://localhost:3306/securevault?createDatabaseIf
spring.datasource.username=root
spring.datasource.password=114114
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA Configuration
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
spring.jpa.properties.hibernate.format_sql=true

# Logging
logging.level.org.springframework.web=INFO
logging.level.com.securevault=DEBUG
```

pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion>

    <parent>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-parent</artifactId>
        <version>3.1.5</version>
        <relativePath/>
    </parent>

    <groupId>com.securevault</groupId>
    <artifactId>secure-vault-server</artifactId>
    <version>1.0.0</version>
    <name>secure-vault-server</name>
    <description>Secure Digital Vault Backend</description>

    <properties>
        <java.version>17</java.version>
    </properties>

    <dependencies>
        <!--SpringBoot web(includes Tomcat + Servlet API)-->
```

```
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-web</artifactId>
        </dependency>
        <!--JPA + Hibernate-->
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-data-jpa</artifactId>
        </dependency>

        <!-- MySQL Driver -->
        <dependency>
            <groupId>com.mysql</groupId>
            <artifactId>mysql-connector-j</artifactId>
            <!--<version>8.3.0</version>-->
            <scope>runtime</scope>
        </dependency>

        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
    </dependencies>

<!--    <build>
        <plugins>
```

```
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>-->
<build>
    <plugins>

        <!-- Spring Boot Plugin -->
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
            <configuration>
                <mainClass>com.securevault.SecureVaultApplication</mainClass>
            </configuration>
        </plugin>

    </plugins>
</build>

</project>
```

Conclusion

The application successfully demonstrates MVC architecture using Spring Boot and Hibernate. It separates concerns into different layers, making the application modular and maintainable. Hibernate simplifies database interaction, while Spring Boot enables rapid development of RESTful services.